

Deliverable 3 – Final Game Prototype & Showcase Report

Replay Ghost Game

Course: Game Design

Submission Date: April 25, 2026

1. Project Overview

Replay Ghost Game is a narrative-driven, mechanic-focused Unity game built around a single central idea: a failed experimental system that lets a test subject interact with past versions of themselves. The player takes the role of a detective sent to investigate a remote forest laboratory where something has gone wrong. Through environmental exploration, NPC dialogue, and progressively more demanding gameplay systems, the player uncovers the full story before facing its source in a direct confrontation.

The design goal was to ground every system—resource management, enemy AI, a final combat encounter—in that one core mechanic: temporal replay. Over three levels the concept grows from a navigation aid into a high-stakes combat constraint, giving the prototype a clear beginning, middle, and end that feel mechanically and narratively coherent.

The final build contains six scenes (Start Menu, Levels 1–3, a Loading Scene, and an End Screen), 26 custom C# scripts, original UI artwork, a curated audio layer, and a complete Windows executable.

2. Art Direction, Aesthetics & Sound Design

2.1 Visual Identity

A deliberate visual shift was made between environments to track the narrative's emotional arc.

Forest / Outdoor Areas (Levels 1 & 2)

Assets from the *Nature Essentials* and *Dungeon Modular Pack* libraries were combined to build an open, slightly unsettling forest. A muted, desaturated colour palette—grey-greens, earth browns, diffuse ambient fog—reinforces the tone of isolation and discovery. The openness of the space allows free exploration while subtle landmark placement guides the player toward objectives without explicit waypoints.

Laboratory and Lair (Level 3)

The final level moves into a darker, enclosed interior. Torch and fire particle effects introduce motion and warm-orange contrast against stone walls, signalling the shift from investigation to confrontation. The tighter geometry creates natural chokepoints that support the duel mechanic.

User Interface

Default Unity UI components were replaced throughout to achieve a style that feels part of the world rather than layered on top of it. The energy bar and health bar use custom Image fills rendered via the `EnergySystem` and `EnemyAttack` scripts respectively. TextMesh Pro handles all narrative and feedback text,

keeping font rendering crisp at any resolution. The combat arrow indicators and the "Final Blow Available" prompt were designed to be legible at a glance without occupying large screen real estate.

2.2 Sound Design

Audio was treated as a functional gameplay layer. Every sound either communicates state information or reinforces an emotional cue.

Context	Sound	Purpose
Forest ambient	Wind, bird loop	Establishes baseline atmosphere, calm before escalation
Level 2 Clock	Ticking audio loop	Auditory countdown reinforces time-pressure mechanic
Level 3 ambient	Fire crackle loop	Spatial grounding, tension in the lair
Exploration → combat	Dynamic music swap (AudioTrigger)	Signals mode transition without UI text
Rewind activation	Whoosh / energy sound	Confirms mechanic input, ties replay to "something happening"
Energy pickup	Chime from Casual Game Sounds pack	Positive reward signal, prompts resource awareness
Block attempt	Male effort grunt (1 of 41 variants)	Immediate acknowledgement of input direction
Successful block	Sword clash (SwordSoundPack)	Positive reinforcement of correct timing
Final blow	Full sword-swing slash	Punctuation on the encounter's conclusion

The *Fantasy Battle Music Free Pack* provides exploration and battle tracks. The AudioTrigger component swaps clips on zone entry, allowing smooth musical transitions between calm exploration and high-intensity combat without hard cuts.

3. Narrative & Systems Integration

3.1 Story Delivery

The narrative is embedded in the environment rather than delivered through cutscenes. Two detective-assistant NPCs provide context and direction at key moments, their dialogue surfaced through NarrativeTrigger volumes that show UI text when the player enters a defined area. Environmental props, lighting choices, and level layout together tell the story between those dialogue beats.

The arc: two detectives arrive at a remote lab, discover an experimental replay apparatus, realise it has been turned against them by the subject who now controls time, and confront that subject in the lair.

3.2 Mechanics as Narrative Expression

Each mechanical shift is also a narrative beat:

Level	Mechanic	Narrative meaning
1	Replay introduced as a found ability	The detectives discover the device
1	Energy limit on replay	The device is unstable, resources are finite
2	Time-loop clock constraint added	The subject is now actively manipulating time
2	Past-version enemies appear	The subject is using the replay system against the player
3	Replay mechanic removed entirely	The device is gone; the player must face consequences directly
3	Directional combat system	A fair skill-based resolution rather than an exploitable tool

The removal of the replay mechanic in Level 3 is the most deliberate narrative-mechanical integration in the project. It is not just a difficulty increase—it is the story's payoff: the player loses their safety net at precisely the moment the stakes are highest.

3.3 World-Building Details

- A park-style forest environment with natural paths and landmark trees creates a sense of a real location rather than an abstract game space.
- Laboratory elements in Level 3 (modular dungeon pack geometry, fire sources, enclosed rooms) contrast sharply with the open-air levels, making the transition feel earned.
- NPC guide character *Spark* follows the player via [SparkGuide.cs](#) waypoints, resets to the start of the current segment when the player rewinds, and disappears in Level 3—one more subtle signal that old rules no longer apply.

4. Progression Design & System Balance

4.1 Three-Act Level Structure

Level 1 – Introduction & Discovery

Objectives: learn movement, encounter the Spark guide, find and use the replay mechanic, reach the lab entrance.

Design: low enemy density, generous energy capsule placement (15 energy per capsule, starting from 100), checkpoints close together. The goal is experimentation without punishment. The [ReplayManager](#) records positions every frame; pressing R costs 20 energy and spawns the ghost, giving tactile proof the system works before demanding anything of the player.

Level 2 – Pressure & Constraint

Objectives: navigate with enemies present, manage the global time limit, reach Level 3.

New elements: [EnemyAI](#) entities patrol and chase on detection; the clock ticks down audibly; energy capsules become scarcer. The 20-energy replay cost and 15-energy capsule gain create deliberate scarcity—players who spam replay run dry. Enemy collision respawns the player at the nearest checkpoint and clears the ghost recording, ensuring replay can't trivially loop around enemies.

Level 3 – Skill-Based Resolution

Objectives: pick up the sword, survive the duel, land the final blow.

Replay is absent. The **EnemyAttack** system drives a four-directional duel:

- Enemy selects an attack direction (0 = left, 1 = right, 2 = up, 3 = down)
- Arrow UI displays the incoming direction
- Player matches it with the corresponding arrow key via **SwordBlock.cs**
- A correct block plays a sword-clash sound; an incorrect block deals 20 damage (max HP 100)
- Enemy attack animation speed scales from 0.4× to 1.0× over the encounter
- At max speed the "Final Blow Available" prompt appears; player presses X to trigger the slow-motion execution and load the End Screen

4.2 Balance Tuning

Parameter	Value	Rationale
Starting energy	100	Generous intro allowance, five free uses
Replay cost	20 energy	Limits abuse; 5 uses before empty
Capsule yield	15 energy	Below replay cost—requires two capsules to restore one use
Enemy move speed	3 m/s	Fast enough to create urgency, slow enough to dodge
Combat HP	100 (5 hits)	Long enough to learn the pattern, short enough to feel consequential
Attack speed range	0.4–1.0	Early phase readable; late phase requires real attention

Energy availability was iterated several times. The original system had no cost, making replay trivial. The first cost iteration (40 energy) with limited capsules created unavoidable failure states. The current 20-cost/15-yield balance maintains tension without frustrating players who engage with the capsule economy.

5. Playtesting & Iterative Development

5.1 Testing Process

Playtesting was conducted in two forms:

1. **Internal iteration** – repeated solo and paired test sessions after each feature addition, targeting specific mechanics (replay feel, enemy behaviour, combat timing).
2. **Peer feedback sessions** – guided play sessions with classmates who had not seen the game, followed by structured debrief questions about clarity, fairness, and frustration points.

5.2 Key Issues Found & Changes Made

Issue Identified	Change Made
Replay mechanic had no limitation, reducing challenge to zero	Introduced EnergySystem ; tuned cost and capsule yield over multiple iterations

Issue Identified	Change Made
Default Unity UI felt disconnected from forest/lab aesthetic	Rebuilt all HUD elements with custom-styled Image fills and TextMesh Pro
Combat timing felt inconsistent—blocks registered too late or too early	Reworked <code>EnemyAttack.cs</code> to check block direction against the animation's progress via <code>animationSpeed</code> -aware timing logic
Enemy speed scaling jumped too abruptly in early versions	Replaced step-function scaling with gradual ramp from 0.4 to 1.0 across the encounter
Players didn't know where to go in Level 2	Added visual landmark placement and NPC directional cues via <code>NarrativeTrigger</code> volumes
Ghost replay animation looked disconnected from world	Tuned ghost prefab position interpolation and added a visual tint to distinguish it from the live player
Scene transitions caused audio to cut abruptly	Moved persistent audio sources to the async <code>LoadingScene</code> controller; <code>AudioTrigger</code> now fades rather than hard-stops

5.3 Debugging & Performance Work

- Resolved animation interruptions during combat that caused the sword-swing to cancel mid-frame
- Stabilised scene transitions: `isLoading` flag in `NextLevelDoor.cs` prevents double-trigger
- Fixed `SparkGuide.cs` waypoint reset not triggering on certain rewind sequences
- Verified that `CheckpointManager` singleton persists correctly across level loads using `DontDestroyOnLoad` semantics (static instance pattern)
- Confirmed Windows build runs without the Unity Editor, with no missing asset references in the final export

6. Technical Implementation

6.1 System Architecture Overview

All custom logic lives in `Assets/Scripts/`. The 26 scripts divide into five responsibility groups:

Group	Scripts
Core mechanic	<code>ReplayManager.cs</code> , <code>GhostReplay.cs</code> , <code>EnergySystem.cs</code> , <code>EnergyCapsule.cs</code>
Combat	<code>EnemyAttack.cs</code> , <code>SwordBlock.cs</code> , <code>SwordPickup.cs</code> , <code>ScreenFadeAndFight.cs</code>
Navigation & world	<code>CheckpointManager.cs</code> , <code>Checkpoint.cs</code> , <code>PressureButton.cs</code> , <code>NextLevelDoor.cs</code> , <code>SparkGuide.cs</code>
Player	<code>PlayerMovement.cs</code> , <code>MouseLook.cs</code> , <code>EnemyAI.cs</code>
UI / Scene	<code>MainMenu.cs</code> , <code>PauseMenu.cs</code> , <code>EndScreen.cs</code> , <code>EndGameMenu.cs</code> , <code>LoadingSceneController.cs</code> , <code>SceneLoader.cs</code> , <code>NarrativeTrigger.cs</code> , <code>AudioTrigger.cs</code> , <code>energybarTrigger.cs</code> , <code>sparkstart.cs</code>

6.2 Replay System

ReplayManager appends **transform.position** to a **List<Vector3>** every **Update()** frame. When the player presses R with sufficient energy:

1. **EnergySystem.UseReplay()** deducts 20 energy and updates the fill bar.
2. **ReplayManager** instantiates the **Ghost** prefab, injects the recorded list into a **GhostReplay** component, and starts playback.
3. The player is teleported back to **CheckpointManager.currentCheckpoint + Vector3.up * 1.5f**.
4. **SparkGuide.ResetSpark()** returns the companion to waypoint 0.
5. The recording list is cleared, beginning a fresh capture from the new position.

6.3 Combat System

EnemyAttack drives a coroutine-based duel loop:

```
ChooseDirection() → Play attack animation at current speed →
Wait for animation hit frame → Check SwordBlock.currentBlockDirection →
    Match → No damage, speed increases slightly
    Mismatch → 20 damage to player health
Speed == maxSpeed (1.0) → Enter Final Blow phase →
    Wait for X input → Trigger slow-motion (Time.timeScale = 0.3f) →
    Play sword-swing animation (120° rotation) → Load EndScene
```

Attack-speed increase is capped at 1.0 (**Mathf.Min**), with the final-blow threshold checked each cycle.

Time.timeScale is restored before **SceneManager.LoadScene** to avoid the End Screen inheriting the slow-motion state.

6.4 Notable Design Patterns

- **Singleton:** **CheckpointManager** exposes a static **instance**, allowing any script to read the current checkpoint position without serialised references.
- **Event-driven audio:** **AudioTrigger** decouples music changes from gameplay scripts; zone entry swaps the clip on the scene's main **AudioSource** without any gameplay script needing audio awareness.
- **Coroutines for sequenced UI:** **ScreenFadeAndFight** chains fade-to-black, character repositioning, fade-in, countdown, and combat-enable into a single readable coroutine sequence rather than a state machine.
- **Async scene loading:** **SceneLoader** + **LoadingSceneController** abstract all scene transitions behind a loading screen, preventing single-frame freezes on heavier scenes.

6.5 Input Mapping Summary

Input	Action
WASD	Player movement
Mouse	Camera look
Space	Jump

Input	Action
R	Trigger replay (costs 20 energy)
Arrow keys	Sword block direction (in combat)
X	Execute final blow (when available)
Escape	Toggle pause menu

7. Conclusion

Replay Ghost Game began as a proof-of-concept for a single mechanic and grew into a structured three-act experience that uses that mechanic to carry both gameplay and story.

The progression from open exploration (Level 1) to constrained navigation under pressure (Level 2) to mechanic-stripped skill combat (Level 3) mirrors the narrative's arc from discovery to confrontation. Each design choice—energy costs, enemy speed ramps, the deliberate removal of replay in the final level—was motivated by what would make the player feel the story rather than just read it.

Iteration through playtesting was essential: the energy system, combat timing, and UI presentation all changed substantially from their first implementations. The final build represents a significantly more polished experience than any prior deliverable—technically stable, visually coherent, and narratively complete.

Assets, Packages, and ProjectSettings directories included in project zip.
Windows build (.exe + Data folder) included and verified functional.